

숫자 야구 게임

2024.06.11.
디지털 시스템 설계 (CSED 273)
20230642 이채영
20230904 조민서

1. 주제 및 개요

본 프로젝트에서 구현하고자 하는 주제는 숫자 야구 게임으로, 플레이어로부터 중복되는 숫자가 없는 세 자리의 수를 입력받아 이 숫자를 정답 숫자로 저장하고 다른 플레이어가 해당 숫자를 맞추도록 하는 게임이다. 구현하려는 장치는 플레이어로부터 중복되지 않는 세 자리의 숫자를 입력받아 해당 숫자를 정답 숫자로 저장한다. 이후 다른 플레이어가 추측한 세 자리 숫자를 입력하면 정답과 입력 숫자의 각 자리수 별로 숫자를 비교하여 값과 위치가 모두 맞는 경우 스트라이크(S, strike), 숫자의 값은 맞지만 위치가 틀린 경우 볼(B, ball), 숫자의 값과 위치가 모두 틀린 경우 아웃(OUT)을 출력한다.

입력받는 숫자는 모두 4 bit 2진수로 변환하여 처리하며 입력되는 숫자와 비교 결과는 FPGA 보드의 7-segment module을 이용하여 확인할 수 있다. 본 프로젝트에서는 숫자 야구 게임의 FSM을 설계하고 Verilog와 FPGA 모듈을 이용하여 실제 숫자 야구 게임을 구현한다.

2. 이론적 배경

1) Karnaugh Map

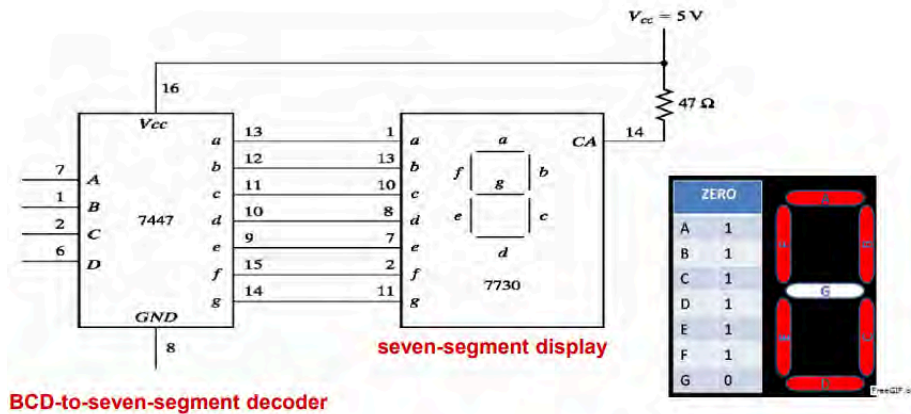
불 대수식을 단순화(Simplification)하면 회로에 사용되는 와이어와 게이트 수를 줄여 소비 전력을 줄이고 작동 속도를 높일 수 있다. Karnaugh Map은 불 대수식을 단순화하는 대표적인 방법으로, 입력과 출력을 도식화하여 0 또는 1의 패턴을 찾아 사각형 묶음으로 단순화한다. 본 프로젝트에서는 BCD로 표현되는 0~9 숫자와 알파벳을 출력하기 위한 BCD to 7-segment decoder를 설계하기 위해 Karnaugh Map을 사용하였다.

2) 4-Bit Magnitude Comparator

4-Bit Magnitude Comparator는 서로 다른 두 개의 4-bit 수가 입력으로 주어졌을 때, 둘의 대소 관계를 판별하여 알맞은 출력을 1로, 나머지 출력을 0으로 설정해 출력하는 회로를 의미한다. 입력값이 A, B로 주어지고 A와 B가 같은 수라고 가정할 경우 출력 $A=B$ 는 1, $A<B$, $A>B$ 는 0이 된다. 본 프로젝트에서는 정답 숫자와 사용자가 입력한 숫자를 비교하기 위해 4-Bit Magnitude Comparator를 사용하였으며, $A=B$ 의 관계만 필요하므로 출력 $A>B$, $A<B$ 는 생략해 구현하였다.

3) 7-Segment Display

7-segment display는 두 개의 사각형 모양으로 7개의 획이 배치되어 있는 디스플레이이다. 7-segment display는 일반적으로 숫자 또는 영문자를 표현하기 위해 사용되며, 본 프로젝트에서는 사용자가 입력한 숫자를 출력하고 비교 결과를 나타내기 위해 해당 디스플레이를 사용하였다.



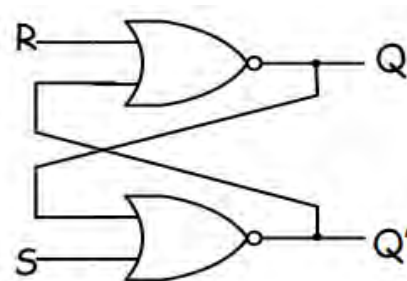
4) Half Adder / Full Adder

Half Adder(반가산기)는 1-bit 이진수 두 개를 입력받아 두 수의 합과 **Carry Out**을 출력하는 장치이다. **Full Adder**(전가산기)는 1-bit 이진수 두 개와 **Carry in**을 입력받아 세 수의 합과 **Carry Out**을 출력하는 장치이다. **Half Adder** 두 개를 이용해 구현할 수 있다. 본 프로젝트에서는 **strike**와 **ball**의 개수를 구하기 위해 **Full Adder**를 이용한다.

5) SR latch

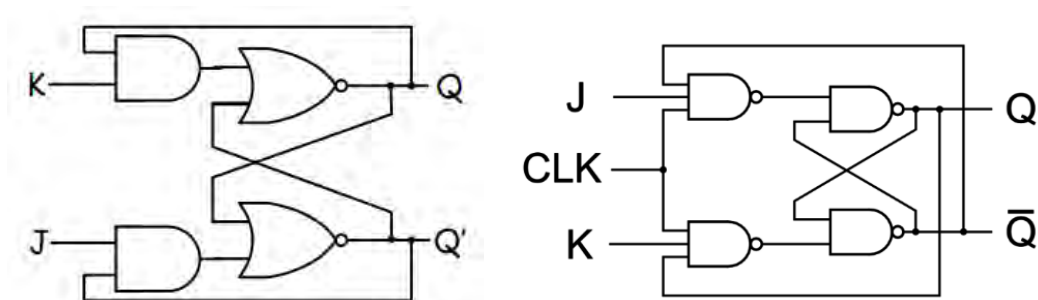
래치(latch)는 1비트의 정보를 보관 혹은 유지할 수 있는 회로이며, **sequential logic**의 기본 요소로 **counter**, **register**, **RAM** 등의 각종 회로를 구성하는 기본 요소이자 **CPU**를 구성하는 밑바탕이 된다. **SR Latch**에서 **S**는 **Set**, **R**은 **Reset**을 의미하며 출력값 **Q**, **Q'**은 서로 **complement** 관계이다.

NOR 게이트로 구성된 **SR Latch**는 **S=1, R=0**일 때는 **Set**, **S=0, R=1**일 때는 **Reset**이 동작된다. **S=0, R=0**이면 출력(**Q**) 상태가 이전과 같게 유지되고, **S=1, R=1**일 때는 두 출력 **Q, Q'**가 모두 0이 되기 때문에 사용하지 않는다.



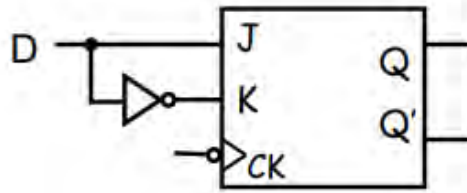
6) JK Latch / JK Flip-Flop

JK Latch는 **SR Latch**에 추가적인 회로를 더해 **S=1, R=1**인 상태에서도 정상적으로 작동되도록 수정한 **Latch**이다. **JK Latch**에서는 **S=1, R=1**일 때 출력 결과가 반전된다. **Latch**는 입력이 바뀔 때 출력이 즉시 바뀌는 비동기 회로인 반면 **Flip-Flop**은 입력이 바뀌더라도 출력이 클럭 (**CLK**) 신호에 맞춰 반영되는 동기 회로이다. 본 프로젝트에서는 **D Flip-Flop**을 구현하기 위해 **JK Flip-Flop**을 사용하였다.



7) D Flip-Flop

D Flip-Flop의 D는 Delay를 의미하며 JK Flip-Flop에서 J와 K 입력이 함께 묶여 있는 Flip-Flop이다. D Flip-Flop은 입력값 D가 Delay되어 출력값에 D가 출력된다. 본 프로젝트에서는 4-bit register를 구현하기 위해 D Flip-Flop을 사용하였다.

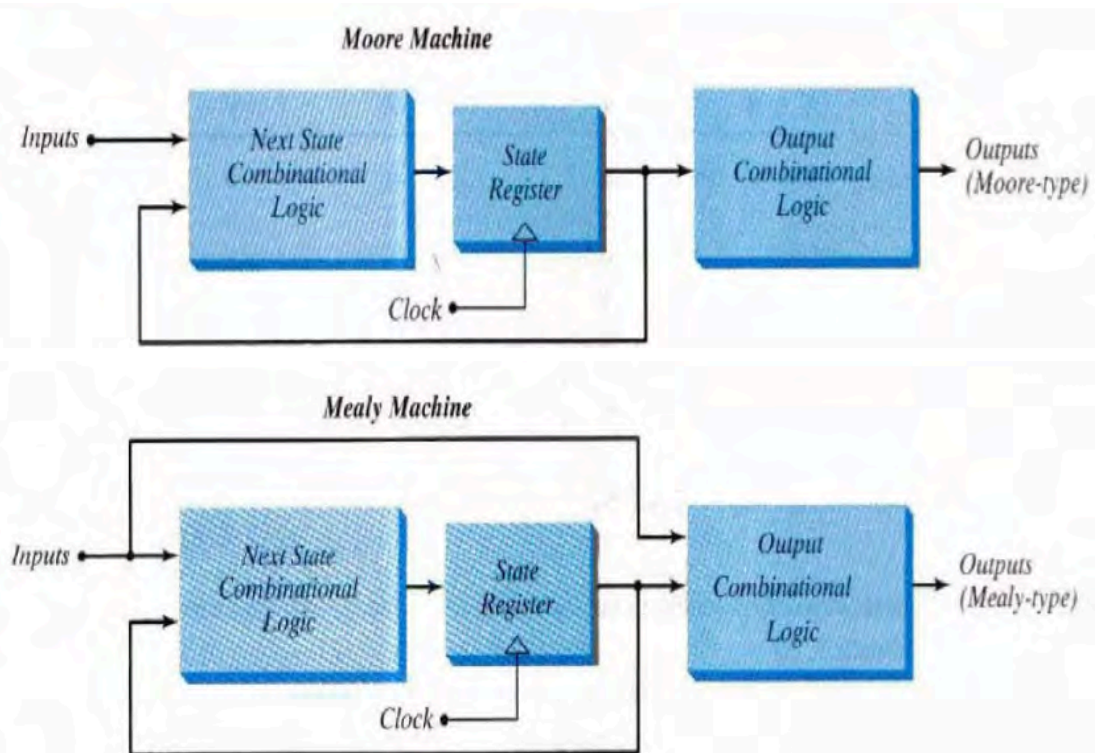


8) Register

Register는 이진 정보를 저장하는 회로로 Flip-Flop을 이용하여 구현된다. 본 프로젝트에서는 4-Bit 이진수를 저장하기 위해 D Flip-Flop 4개를 이용하여 Register를 구현하였다.

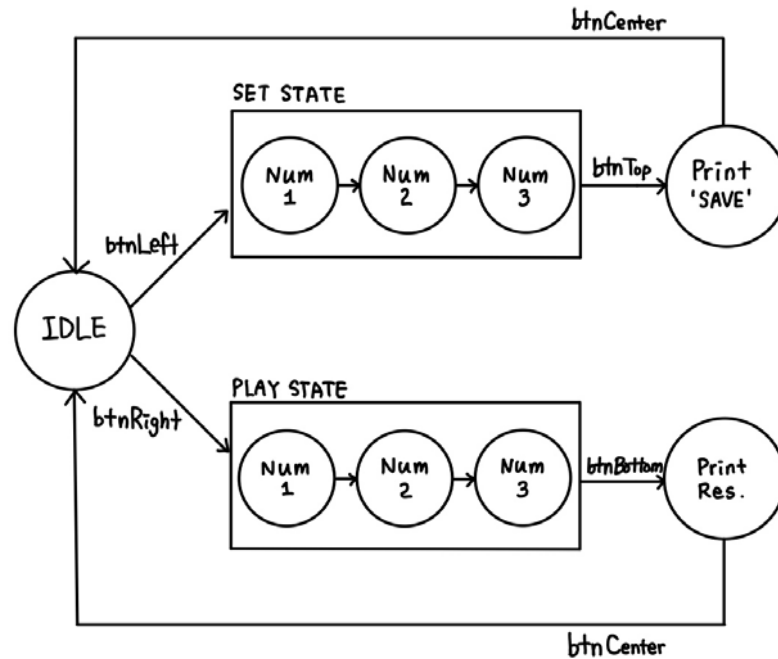
9) FSM

FSM(Finite State Machine)은 유한한 개수의 State를 가지는 Sequential Circuit이다. FSM은 Moore Machine과 Mealy Machine으로 나뉘어진다. Moore Machine은 출력값이 현재 상태에 대한 함수로 나타나며, 활성화된 Clock edge와 동기화되게 상태 전이한다. Mealy Machine은 출력값이 현재 상태와 입력값에 대한 함수로 나타나며, 입력값이 변화하자마자 출력값이 변화한다.



3. 설계 및 구현

1) State Transition Diagram 및 FSM 동작



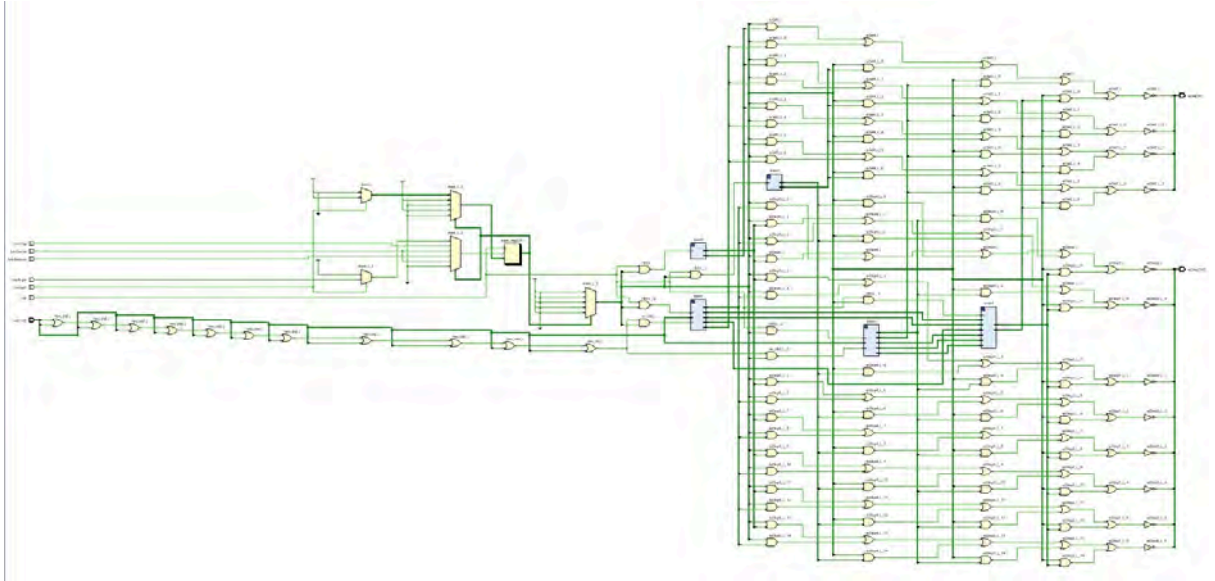
구현하고자 하는 숫자 야구 게임의 State transition Diagram은 위와 같다.

FPGA 보드를 처음 Power On 하면 IDLE state에 진입하고 7-segment module에 0000이 출력됨을 확인할 수 있다. IDLE state에서 FPGA 보드의 왼쪽 버튼을 누르면 Set state로 진입한다. Set state에서는 사용자로부터 정답을 입력받는다. 사용자는 FPGA 보드의 스위치를 이용하여 BCD로 된 세 자리 숫자를 입력한다. 사용자가 스위치를 조작함에 따라 즉시 7-segment module을 이용하여 정답 숫자를 확인할 수 있다. 세 자리 숫자를 입력하면 FPGA 보드의 위쪽 버튼을 눌러 Print SAVE state로 진입한다. 이 state에서는 7-segment module에 'SAVE'가 출력됨을 확인할 수 있고, FPGA 보드의 가운데 버튼을 눌러 IDLE state로 진입할 수 있다. Set state는 재진입할 수 있으며, 재진입할 경우 사용자가 새로 스위치 조작을 통해 새로운 정답을 입력할 수 있다.

IDLE state에서 FPGA 보드의 오른쪽 버튼을 누를 경우 Play state로 진입한다. Play state는 사용자가 정답을 맞추는 단계로, Set state와 마찬가지로 FPGA 보드의 스위치를 이용하여 BCD로 된 세 자리 숫자를 입력하고 7-segment module에서 입력한 숫자를 확인할 수 있다. 세 자리 숫자를 모두 입력하면 FPGA 보드의 아래쪽 버튼을 눌러 Print Res state로 진입할 수 있다. Print Res state에서는 사용자가 입력한 숫자와 정답을 비교한 결과를 7-segment module에 출력한다. 비교 결과는 OUT, 1B, 2B, 3B, 1S, 2S, 1B1S, 2B1S, 3S로 총 9가지가 있다. 이후 FPGA의 중앙 버튼을 누르면 IDLE state로 진입한다. Play state 또한 재진입할 수 있으며, Set state에 재진입하여 정답 숫자를 바꾸기 전까지 같은 정답 숫자가 유지되어 정답을 비교할 수 있다.

2) FSM 전체 회로 구조도

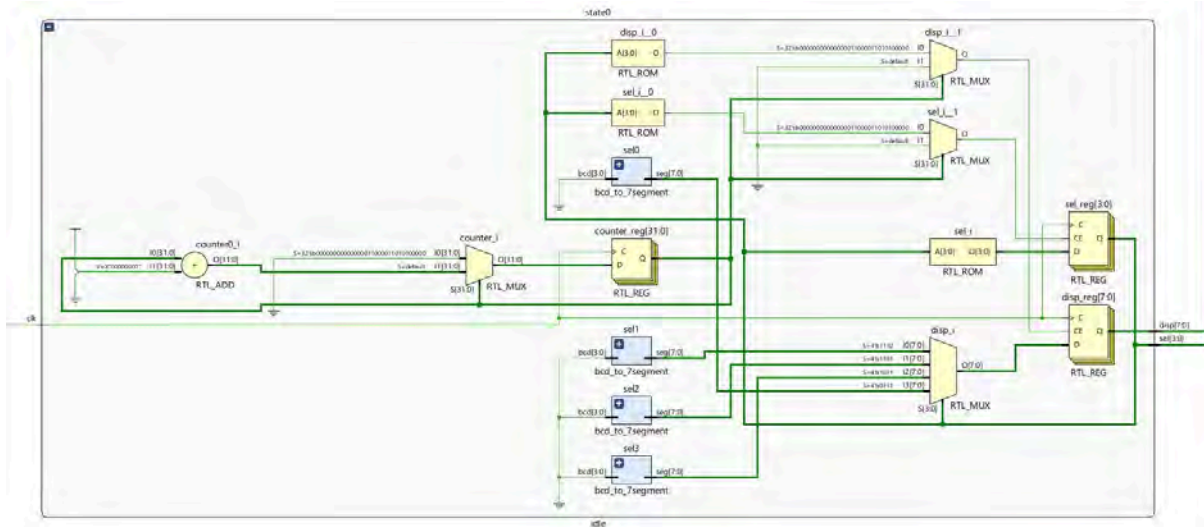
Schematic으로 확인한 숫자 야구 게임의 전체 회로도 는 아래와 같다.



3) FSM 세부 모듈 설계

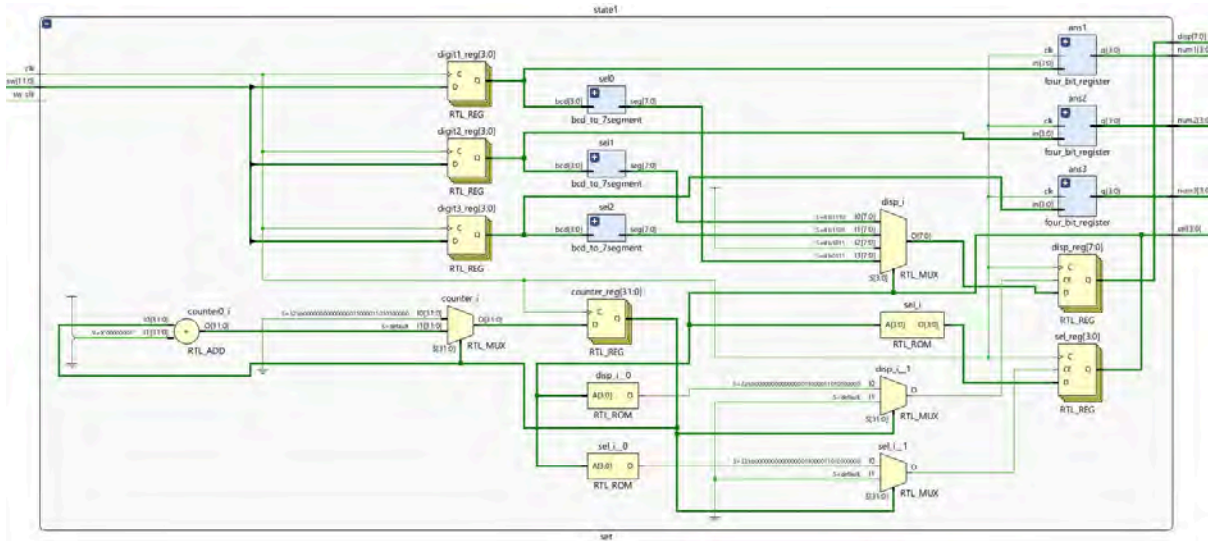
(1) IDLE state

7 segment display에 0000을 출력하고 버튼 입력에 따라 Set state 또는 Play state로 전환한다.



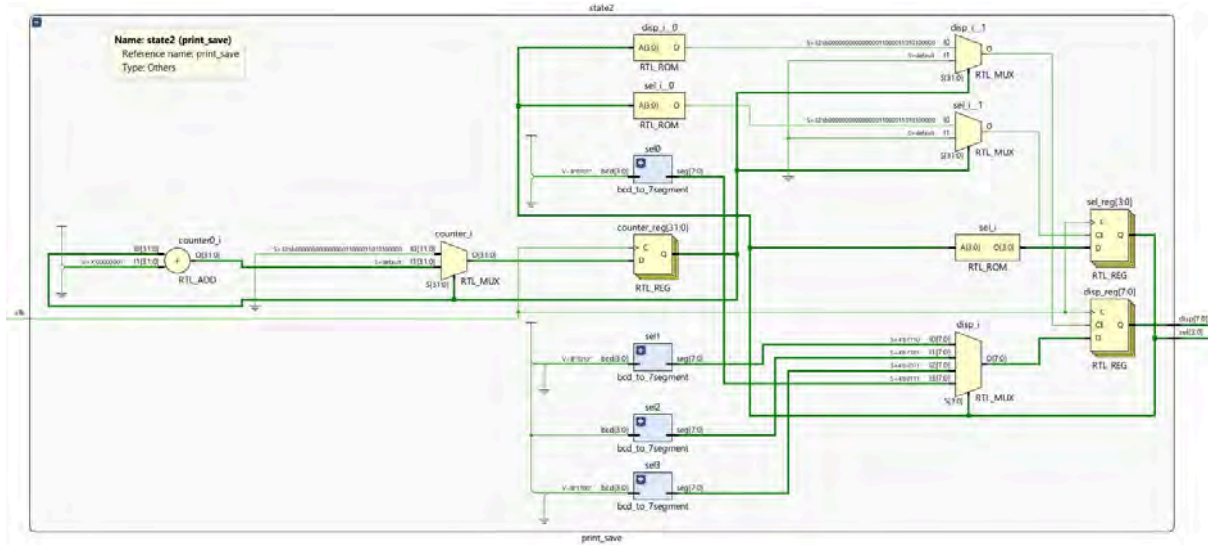
(2) Set state

사용자가 스위치 조작을 통해 입력한 정답 숫자 세 자리를 4-bit Register 3개에 각각 저장한다. 7 segment display에 4-bit Register에 저장된 BCD 숫자를 decimal digit으로 변환하여 출력한다. 버튼 입력에 따라 Print Save State로 전환한다.



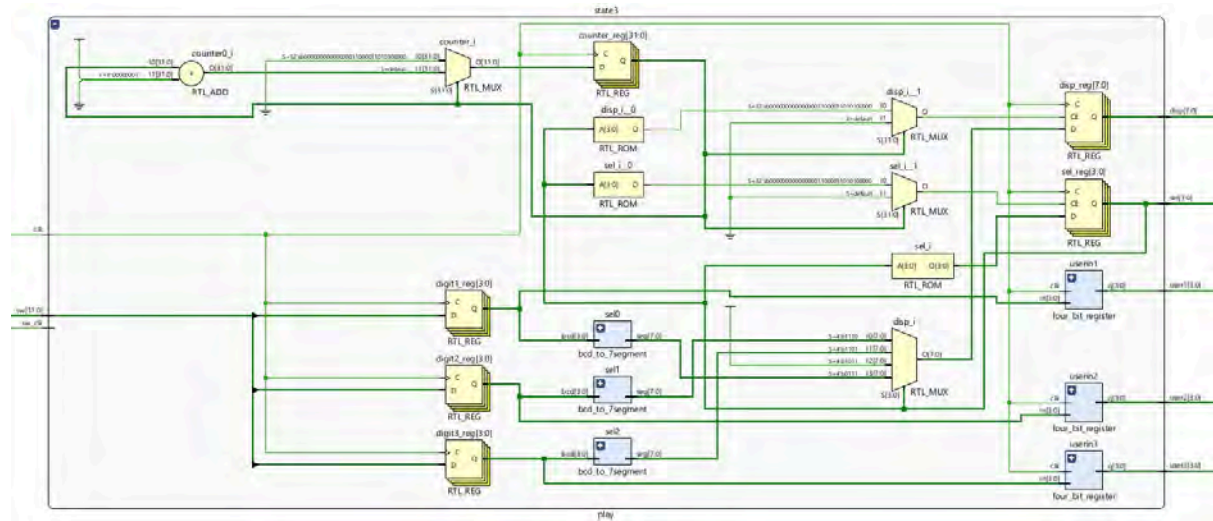
(3) Print Save state

7 segment display에 'SAVE'를 출력하고 버튼 입력에 따라 IDLE state로 전환한다.



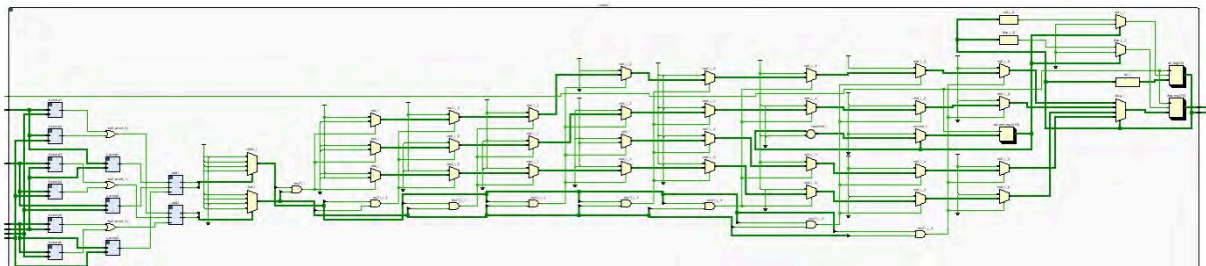
(4) Play state

사용자가 스위치 조작으로 입력한 숫자 세 자리를 4-bit Register 3개에 각각 저장한다. 7 segment display에 4-bit Register에 저장된 BCD 숫자를 decimal digit으로 변환하여 출력한다. 버튼 입력에 따라 Print Res State로 전환한다.

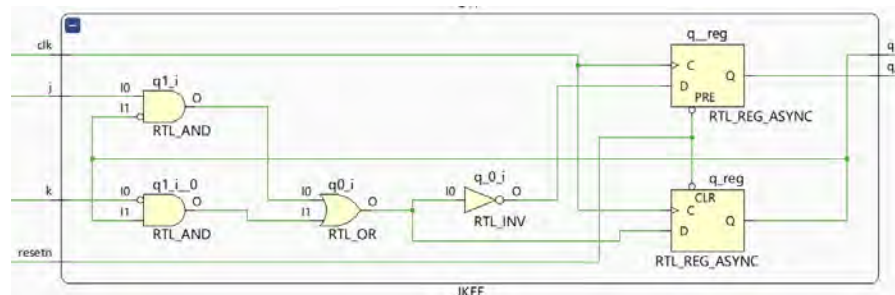


(5) Print Res state

Set state에서 입력한 정답 숫자와 Play state에서 입력한 숫자를 비교하여 strike와 ball의 수를 결정한다. FullAdder를 이용하여 총 strike와 ball의 수를 구한 뒤 각 경우에 따라 7-segment display에 비교 결과를 출력한다.

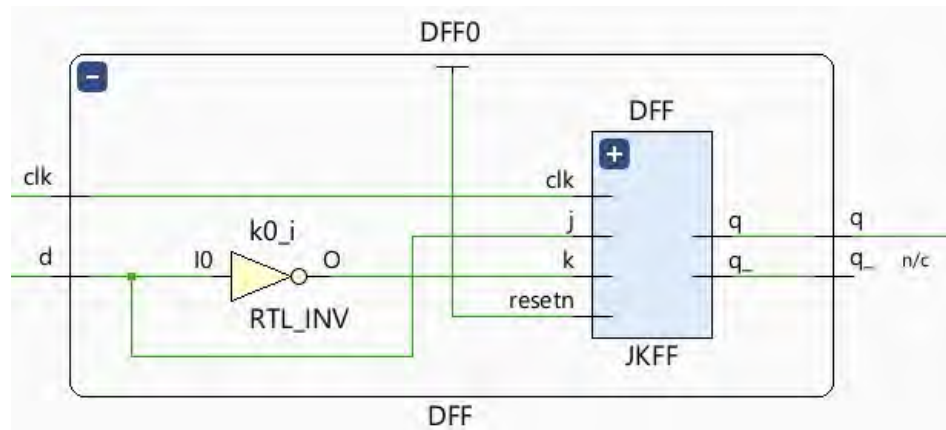


(6) JK FF



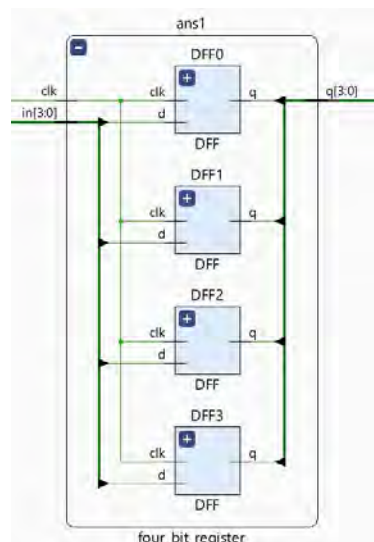
(7) D FF

JK FF의 J, K 값에 D, D'을 입력하여 D FF을 구현하였다.



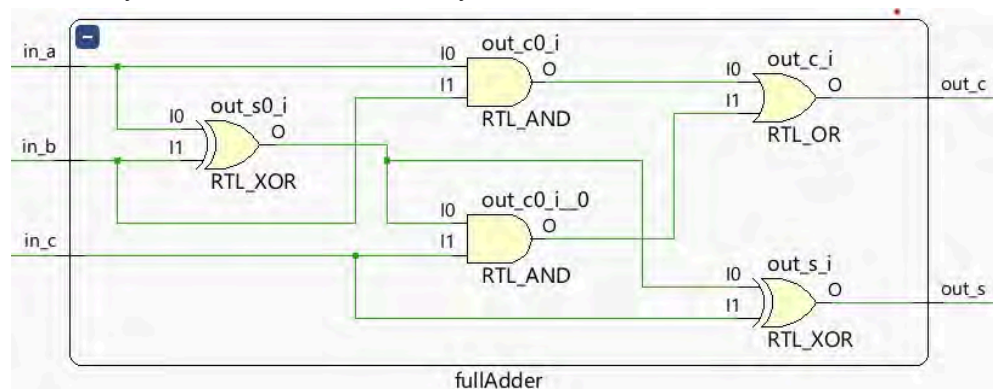
(8) 4-Bit Register

D FF 4개를 연결하여 4-Bit 이진수를 저장할 수 있는 4-Bit Register를 구현하였다.



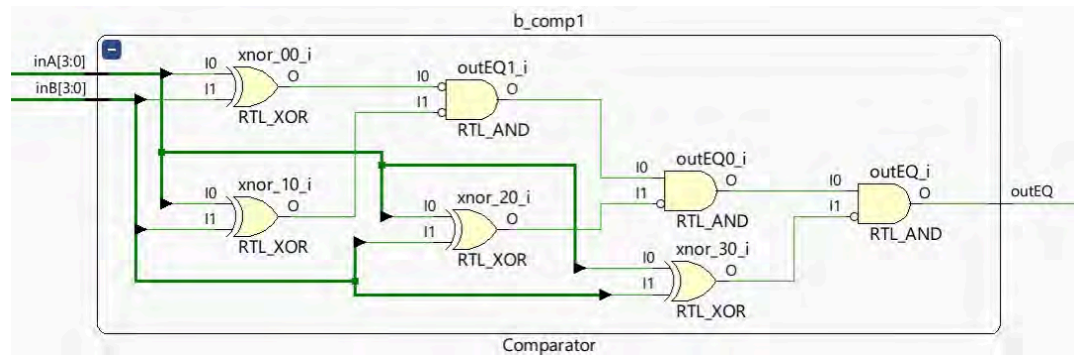
(9) Full Adder

A, B, Carry in 입력에 따라 Sum과 Carry Out을 출력하는 Full Adder 모듈을 구현하였다.



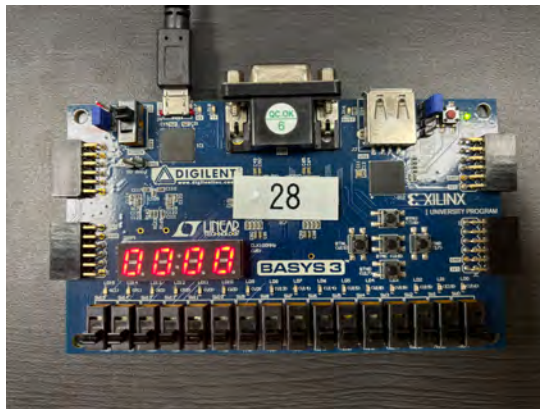
(10) 4-Bit Comparator

4-Bit 이진수 두 개를 입력으로 받아 두 수가 같을 경우 1을, 같지 않을 경우 0을 출력하는 4-Bit Comparator를 구현하였다.

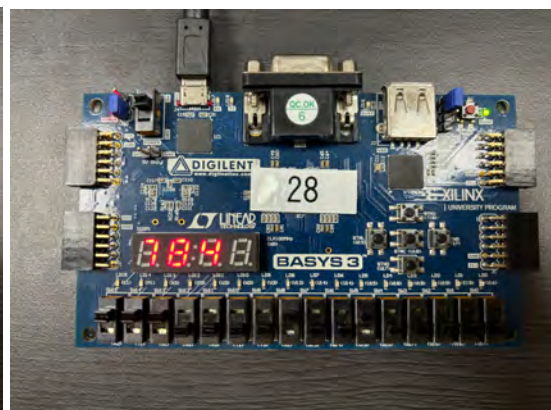
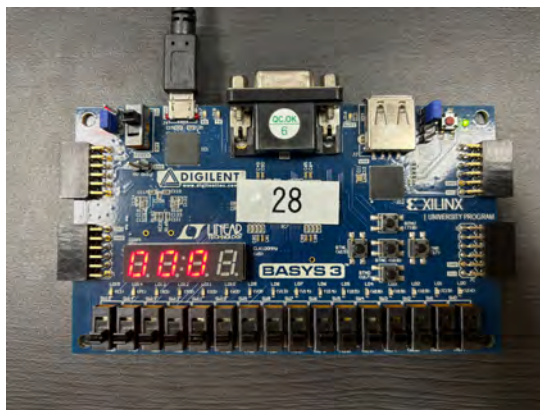


4. 결과

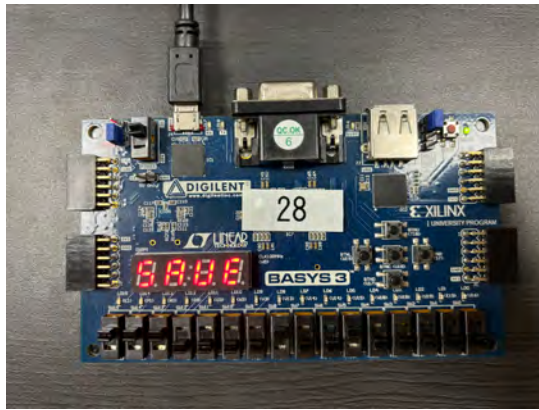
1) IDLE state



2) Set state

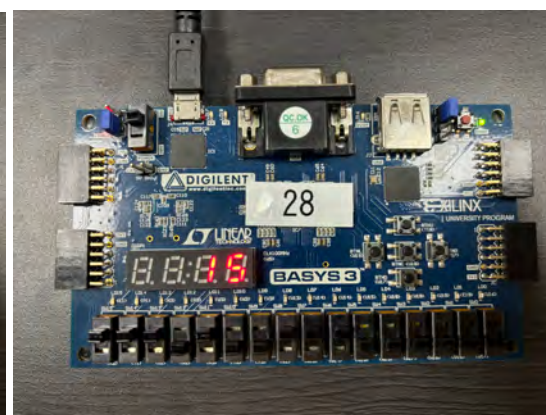
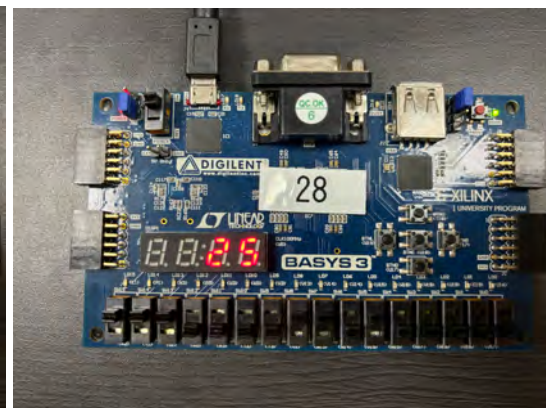
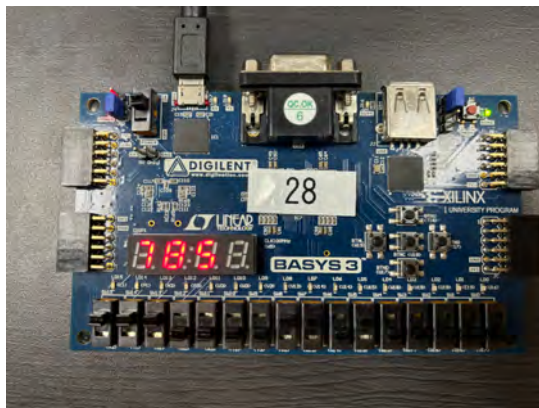
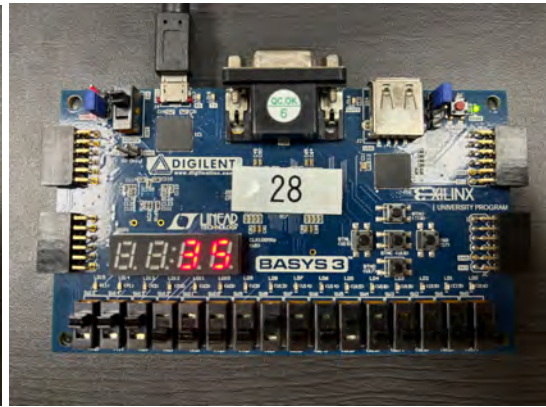
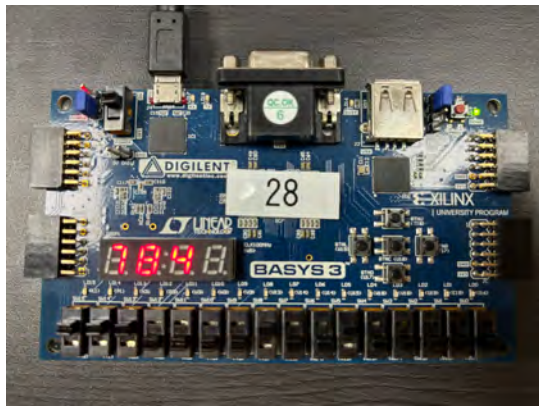


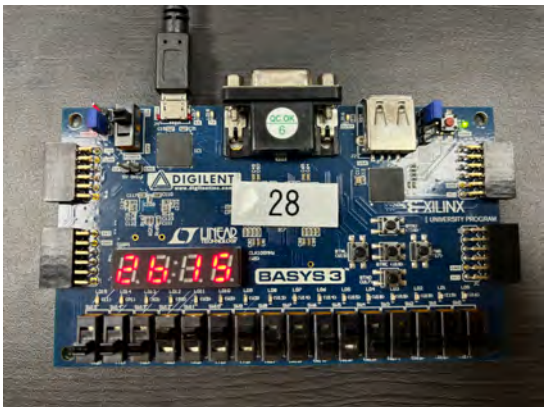
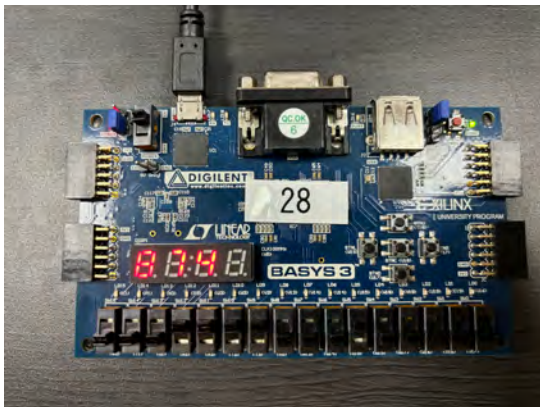
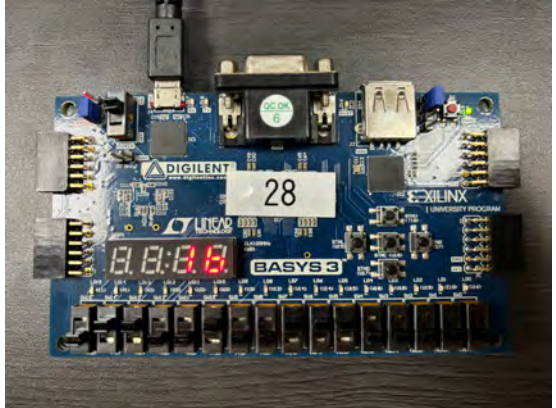
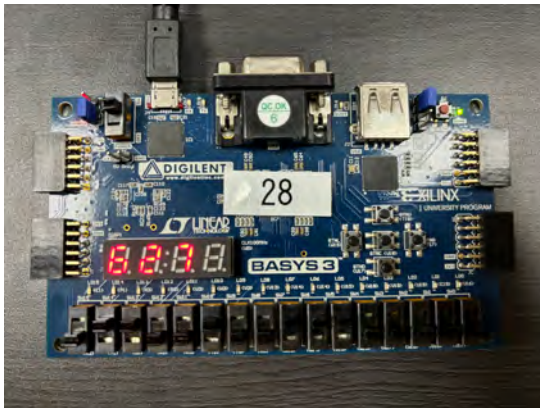
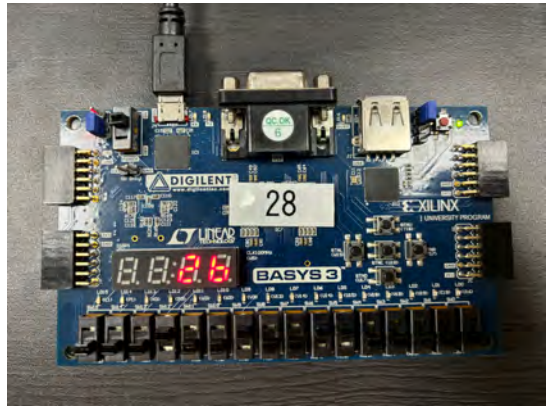
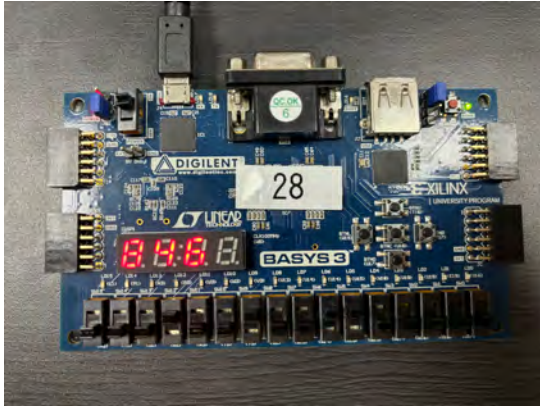
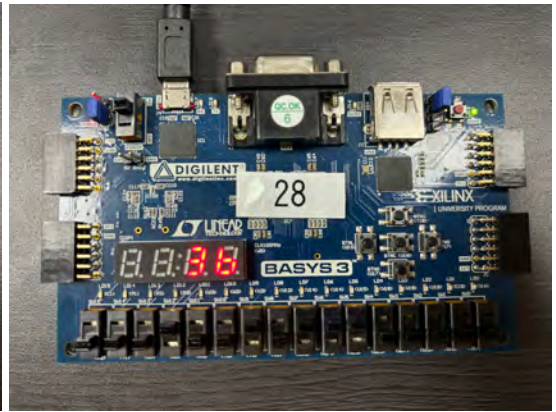
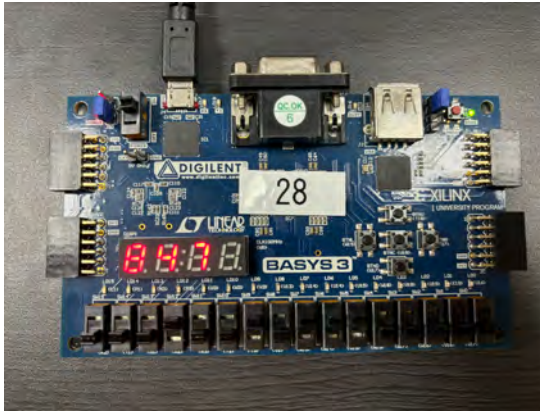
3) Print Save state

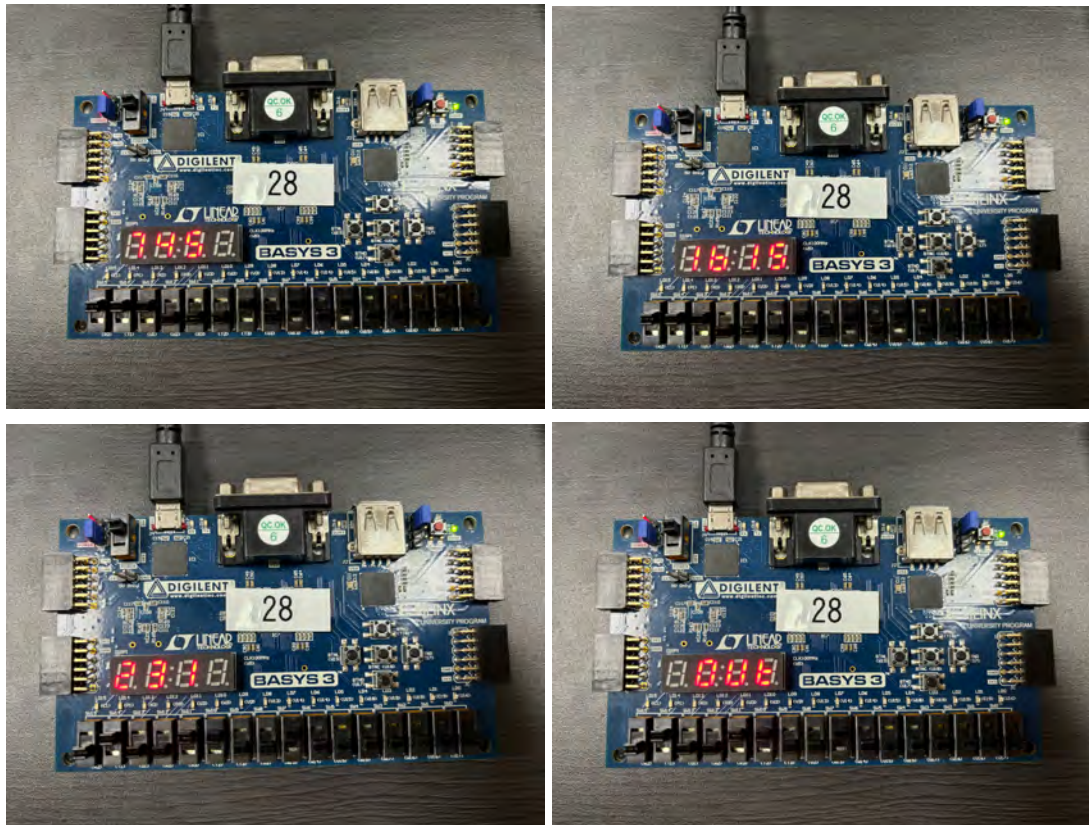


4) Play state & Print Res state

Play state에서의 각 입력(왼쪽)에 따라 Print Res state에서 출력되는 결과(오른쪽)







5. 논의

1) 어려웠던 점 및 해결 방법

Verilog로 코드를 짜면서 모듈을 인스턴스화하지 않아서 오류가 발생했고, **always** 구문 안에 모듈 인스턴스화 구문을 넣어서 오류가 발생하기도 했다. 이를 통해 모듈을 사용하기 위해서는 반드시 인스턴스화를 해야 한다는 점을 알게 되었고, **always** 구문의 활용 방법을 알게 되었다.

프로젝트 **Proposal** 과정에서는 **set state**와 **play state**에서 숫자를 입력받을 때 키패드를 연결하여 입력하는 것으로 계획하였다. 그러나 구현 과정에서 키패드는 눌렀을 때는 숫자가 출력되다가 손을 떼면 숫자가 출력되지 않는 문제가 발생하였다. 또한 키패드는 두 버튼을 동시에 입력하는 경우를 처리하기가 어려운 문제점이 있었다. 따라서 **FPGA** 설계에 간단하게 통합될 수 있고, 단일 입력만 고려해도 되도록 숫자 입력을 스위치로 받기로 변경하였다.

프로젝트를 구현하면서 가장 어려웠던 부분은 클럭을 적용하는 부분이었다. **set state**와 **play state**에서 스위치로 숫자 3개를 입력하면 입력한 값이 **FPGA 7segment**로 출력이 되어야 했는데 계속 출력이 되지 않는 문제가 발생했다. 이 문제는 **sw_clk**(스위치 클럭)과 **clk**을 구분하고 입력한 4-bit digit을 7segment로 변환하는 데 **posedge clk**(positive edge clk) 신호를 사용함으로써 해결할 수 있었다.

FSM에서 **State** 간에 전환될 때, 기존 **Lab**의 코드를 참고하여 작성하였지만 이는 **Gate-Level Modeling**이 아님을 알게 되었다. 이에 **T FF**를 활용하여 **State** 전환을 할 수 있도록 구현하고자 했으나 시간적 한계로 인해 구현하지 못하였다.

2) 추가 구현 가능성

지금 프로그램에서는 사용자가 **set state**와 **play state**에서 자릿수끼리 서로 중복되지 않는 수를 입력하지 않는다고 가정한다. 만약 중복되는 숫자가 입력되면 스트라이크와 볼 카운팅이 제대로 되지 않아 **Out**이 출력되는데, 이 경우 에러 메시지를 출력하는 **state**로 넘겨 구현해볼 수 있을 것 같다. 또한 **State** 전환 기능을 **Gate Level Modeling**으로 구현하는 것을 시도해볼 수 있을 것 같다.